- block-emulator调用关系
  - main.go
    - test.Test_shard()
      - 38.if isClient:pbft.RunClient(testFile)
        - pbft/client.go/RunClient(path string)
          - 77.if params.Config.ClientSendTX
            - 78.txs = Get_Initial_Map_And_TXS(path, account.Account2Shard): 从CSV文件中读取交易数据，并将这些交易数据存储在Transaction中，并将账户地址映射到分片的映射关系存储在Account2Shard映射中
              - pbft/client.go/775.Get_Initial_Map_And_TXS(path string, Account2Shard map[string]int) []*core.Transaction
            - 177.SendNewAddr2Shard(new_addrs, new_addr2shard)
              - cNewMap--->工作分片主节点
                - pbft/pbft.go/handleRequest(data []byte)/203.case cNewMap:go p.handleNewMap(content)
                  - 1714.if params.Config.Stop_When_Migrating
                    - 1722.p.mpropose(new_addr2shard)//直接epochchange pbft/PBFTforMigrate.go/135.mpropose(new map[string]int)
                      - 159.p.commit1(content, pbftType := "EpochChange") pbft/pbftsingle.go/commit1(content []byte, pbftType string)
                        - if p.Node.nodeID == "N0"
                          - 337.pbft/PBFTforMigrate.go/go p.SendOut(out)整理要发出去的账户的状态和交易，并发到对应分片主节点
                            - cBalanceAndPending--->目标分片主节点
                              - pbft/pbft.go/handleRequest(data []byte)/216.case cBalanceAndPending:p.handleBalancesAndPendings(content)
                                - 352.p.spropose(inacc_pool)
                                  - pbft/PBFTforMigrate.go/188.p.commit1(content, pbftType:= "AccState")
                                    - else修改账户状态树，将新账户放入account.AccountInOwnShard中，并设为true，将交易放入交易池
                                      - p.SendSure()//遍历params.NodeTable，向每个分片的主节点发送cSure消息
                                        - pbft/PBFTforMigrate.go/390.handleSure(content []byte)每次收到一个sure消息，sure_count变量加1.当sure_count等于配置中的分片数量params.Config.Shard_num时，表示收到了所有分片的sure消息。在这个条件下，重置sure_count为0，并解锁epochLock，允许进行下一个epoch的操作
                                - 361.if flag && inacc_count == 0////有要进来的，且进完了
                                  - p.spropose(inacc_pool)
                                - 364.else if !flag////没有要进来的
                                  - go p.SendSure()
                  - 1723.else
                    - 寻找本分片在epoch要迁出的账户 1728.p.Node.CurChain.TXmig1_pool.AddTXmig1(&core.TXmig1{Address: addr, FromshardID: params.ShardTable[params.Config.ShardID], ToshardID: new_addr2shard[addr], Request_Time: time.Now().UnixMilli()})
                      - core/txmig1pool.go/23.AddTXmig1(txmig1 *TXmig1)
            - 211.go InjectTXS(txs)
              - cClient--->工作分片主节点
                - pbft/pbft.go/handleRequest(data []byte)/199.case cClient:go p.handleTxFromClient(content)处理从客户端接收到的交易（cClient），并根据节点的配置和分片信息，将交易添加到节点的交易池中。还处理了跨分片交易tx2的情况，确保交易能够正确地发送到目标分片
                  - pbft/pbft.go/1093.p.TrySendTX(tx2):cClient--->工作分片主节点
          - 182.else
            - Get_Initial_Map(path, account.Account2Shard)
      - 125.node = shard.NewNode()
        - shard/shard.go/28.NewNode() *Node启动节点
          - pbft/pbft.go/93.NewPBFT()
          - if !config.ClientSendTX
            - node.P.Node.CurChain.Tx_pool.NewInjectTxs2Shard(params.ShardTable[config.ShardID])
          - else
            - 由客户端发送交易
          - pbft/pbft.go/228.Propose()生成一个区块并发起共识
            - pbft/pbftsingle.go/24.p.propose1()
              - pbft/pbftsingle.go/p.commit1(encoded_block, pbftType:= "Block")
                - outbalance := p.Node.CurChain.AddBlock(block) //返回要迁出账户的余额
                - pbft/handleanns.go/p.handleAnns(block.Anns, st1, migTree)工作分片节点处理TXanns，包括更新账户状态、收集相关交易、计算余额变化
                  - go p.mpAndSend(isdelete, anns, cAps, st, migTree)将所有涉及账户的余额变化和pending交易发送给目标分片，并将交易池中的交易发给client
                    - pbft/pbft.go/p.TrySendChangesAndPendings(nss[shard], caps, shard)源分片将所有涉及账户的余额变化，pending交易以及新状态迁移交易TXns发送给目标分片
                      - cCaP--->目标分片主节点
                        - handleChangesAndPendings(content []byte)目标分片主节点收到cCaP并处理，将pending放入交易池，将changes放入change池，并将ChangeAndPending发给client
                          - cNumOfMigratedTXsAddrs--->客户端
                            - pbft/client.go/handle(data []byte)/case cNumOfMigratedTXsAddrs:更新未完成迁移的交易数量num_of_unfinished_migrated_TXs
                    - cUnchangedState--->客户端
                      - pbft/client.go/handle(data []byte)/case cUnchangedState:（Fine-Tuned中，账户余额从源分片到目标分片未变化的）：更新未完成迁移的数量num_of_unfinished_migration
                - pbft/pbft.go/go p.TryRelay(relaytxs, block.Transactions, st2, p.sequenceID)构建交易树和状态树，生成交易和状态的证明。将交易分配到对应的relay池中。将relay交易打包成消息，并通过TCP连接发送给目标分片的主节点
                  - cRelay--->对应的工作分片主节点
                    - handleRelay(content []byte)处理从其他分片接收到的relay交易，如果接收者属于当前分片，则将交易添加到本地交易池中，如果接收者不属于本分片了，则调用TryRelay2(relaytx2)方法转发relay交易转发给目标分片
                      - p.TryRelay2(relaytx2)将relay交易（不带证明）发送给目标分片的主节点
                        - cRelay--->对应的工作分片主节点
                - pbft/pbft.go/p.TryTXmig1(block.TXmig1s, outbalance, st3, migTree)验证TXmig1交易的有效性，并将TXmig2发送给目标分片的主节点
                  - cTXmig1--->对应分片主节点
                    - handleMig2(content []byte)处理从其他分片节点接收到的迁移请求（cTXmig1），并将其添加到当前节点的迁移交易池TXmig2_pool中
                - pbft/pbft.go/go p.TryAnnounce(block.TXmig2s, st4, migTree)验证TXmig2交易的有效性，向其他分片的主节点发送TXann消息
                  - cAnnounce--->客户端和其他分片主节点
                    - pbft/client.go/handle(data []byte)/case cAnnounce客户端更新账户到分片的映射Account2Shard
                    - pbft/pbft.go/handleRequest(data []byte)/case cAnnounce
                      - handleAnnounce(content []byte)源分片和第三方分片处理目标分片发送的cAnnounce消息，并根据通知内容更新本地交易池和账户信息
                - cReply--->客户端（主节点向客户端发送已确认上链的交易集）
                  - pbft/client.go/handle(data []byte)/case cReply：解析内容为Txs_and_Num_of_New_State结构。更新未完成迁移的数量和迁移的交易数量。如果所有迁移完成，设置最大提交块。更新零块计数，如果连续收到5个空块，停止系统运行。更新提交交易集合，并处理迁移逻辑
            - p.propose()